Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

DataFlow AI: A Stacked, Confidence-Aware Framework for Enterprise Data-Integrity Validation and Cross-Dialect SQL Migration on Real Public Datasets

UPASANA BHAUMIK[1], M. A. BERLIN[2]
[1,2]School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT), Vellore, Tamil Nadu 632 014, India
Corresponding author: M. A. Berlin (e-mail: berlin.ma@vit.ac.in).

ABSTRACT Enterprise data platforms continue to lose substantial value to poor data quality, and database-migration projects routinely overrun because integrity validation and query translation are still treated as disconnected problems. We present DataFlow AI, a deployment-oriented framework that couples a stacked anomaly-detection ensemble with an audit-grade, cross-dialect SQL transpilation pipeline, and that evaluates both halves under a single reproducibility contract. The detector fuses four heterogeneous base signals—rule-based domain constraints, robust per-group statistics, an Isolation Forest, and a Local Outlier Factor—through a logistic-regression stacker trained on five-fold out-of-fold predictions, replacing the fixed-weight late fusion of earlier designs. Rather than reporting a single synthetic benchmark, we evaluate end-to-end on three real public corpora: U.S. SEC EDGAR financial-statement values, the New York City FY2024 citywide payroll, and the UCI credit-default corpus, into each of which we inject fifteen reproducible anomaly families at a realistic prevalence of about five percent. On a fixed seed, the stacked ensemble attains an F1 of 0.511 on the SEC corpus (an absolute gain of 0.043 over the strongest single detector), 0.359 on payroll, and 0.717 on credit default, with per-family recall above 0.94 on five of fifteen families and below 0.10 on three families whose discriminative signal we deliberately omit from the feature set—an honest decomposition that synthetic studies tend to obscure. End-to-end throughput on the 200,000-row payroll corpus is 17.3 s on a single CPU core, and runtime grows linearly in the number of rows. For migration, we benchmark transpilation on 109 curated queries across five dialects (PostgreSQL, MySQL, T-SQL, Oracle, and Snowflake), totalling 575 source–target pairs. Parse and transpile success both exceed 0.99, while a stricter abstract-syntax-tree (AST) footprint-equivalence test yields 0.800 overall and degrades from 0.921 on easy queries to 0.738 on hard ones. We treat that gap as the substantive finding: surface translation is essentially solved by mature parsers, but preserving semantics across dialects with divergent windowing, pivot, and lateral constructs remains open. The contribution is therefore not a new accuracy record but a transparent, reproducible demonstration that deployment-aware, confidence-coupled integration of validation and migration is both feasible and operationally sensible. All code, anomaly masks, and SHA-256-verified datasets reproduce end-to-end from a fixed seed.

INDEX TERMS Data integrity, anomaly detection, stacked ensembles, query transpilation, cross-dialect SQL migration, confidence-aware routing, AST equivalence, trustworthy AI, enterprise data quality, reproducibility

I. INTRODUCTION

A. DATA QUALITY AND MIGRATION AS A COUPLED PROBLEM

Modern organisations operate increasingly heterogeneous data estates, and two recurring failure modes dominate their

TABLE 1: List of Abbreviations and Notation

Symbol	Definition	Symbol	Definition
Abbreviations			
AI	Artificial Intelligence	API	Application Programming Interface
LLM	Large Language Model	DB	Database
SQL	Structured Query Language	ML	Machine Learning
IF	Isolation Forest	LOF	Local Outlier Factor
LR	Logistic Regression	OOF	Out-Of-Fold
AST	Abstract Syntax Tree	MAD	Median Absolute Deviation
F1	F1 Score (Harmonic Mean)	FPR	False Positive Rate
TP	True Positives	FP	False Positives
FN	False Negatives	TN	True Negatives
AUC-PR	Area Under Precision–Recall Curve	CV	Cross-Validation
DDL	Data Definition Language	DML	Data Manipulation Language
CTE	Common Table Expression	CPU	Central Processing Unit
SEC	Securities and Exchange Commission	UCI	Univ. of California, Irvine (repository)
ERP	Enterprise Resource Planning	BI	Business Intelligence
Data, Labels, and Indices			
$\mathcal{D}$	Input dataset, $\mathcal{D} = \{x_i\}_{i=1}^{n}$	n	Number of rows in $\mathcal{D}$
x_i	Feature vector / record for row i	y_i	Anomaly label of row i ($y_i = 1$ denotes anomaly)
y	Vector of row labels, $y \in \{0,1\}^n$	i	Row index, $i \in \{1, \ldots, n\}$
Stacked Ensemble (Algorithm 1, Eq. (1))			
f_k	k-th base detector	k	Base-detector index, $k \in \{1,2,3,4\}$
$s_i^{(k)}$	Base score from f_k on row i, $s_i^{(k)} \in [0,1]$	S	Matrix of base scores, $S \in [0,1]^{n\times 4}$
$\mathbf{w}$	Learned stacker weight vector, $\mathbf{w} \in \mathbb{R}^4$	w_k	Weight on base detector f_k
b	Bias term of the stacker	$\hat{s}_i$	Fused (stacked) score for row i, $\hat{s}_i \in [0,1]$
$\hat{s}$	Vector of fused scores, $\hat{s} \in [0,1]^n$	$\mathbf{0}_n$	Zero vector of length n
$\sigma(z)$	Logistic sigmoid, $\sigma(z) = 1/(1+e^{-z})$	z	Argument of $\sigma(\cdot)$
F_j	j-th stratified cross-validation fold	j	Fold index, $j \in \{1, \ldots, k\}$
k (folds)	Number of stratified folds ($k = 5$)	tr	Training index set, $\{1, \ldots, n\} \setminus F_j$
$S_{\text{tr}}, y_{\text{tr}}$	Training rows of S and corresponding labels	$\mathbf{w}_j, b_j$	Fold-j logistic-regression parameters
$S_{F_j}, \hat{s}_{F_j}$	Restriction of S and $\hat{s}$ to fold F_j	`class_weight`	Logistic-regression class weighting (`balanced`)
Robust-Statistics Base Detector			
z_i	Robust per-group z-score for row i	g	Group index for per-group statistics
med_g	Within-group median	MAD_g	Within-group median absolute deviation
1.4826	MAD-to-σ consistency constant	`contamination`	Isolation Forest prior (`'auto'`)
Threshold Selection and Routing			
τ	Decision threshold on $\hat{s}$	τ^*	Best-F1 threshold (per dataset)
$F_1(\tau)$	F1 score as a function of τ	$\arg\max_\tau$	Argument τ that maximises the expression
Cross-Dialect SQL Transpilation (Algorithm 2)			
Q_{src}	Source SQL query	Q_{tgt}	Target SQL query
D_{src}	Source SQL dialect	D_{tgt}	Target SQL dialect
A	AST of source query Q_{src}	A'	AST of re-parsed target query Q_{tgt}
t_0	Wall-clock timestamp at call start	*parse, trans, equiv*	Boolean outcome flags of the pipeline
Functions			
NOW()	Returns the current wall-clock timestamp	PARSE(Q, D)	Parse query Q in dialect D and return its AST
A.TOSQL(D)	Render AST A as SQL in dialect D	FOOTPRINT(A)	AST footprint: node-type counts + table set + column set
SHA256$(\cdot)$	SHA-256 cryptographic hash function	int$(\cdot)$	Integer cast of a hex/byte string
Datasets, Anomaly Families, and Detector Variants			
D1	SEC EDGAR financial-statement corpus	D2	NYC payroll FY2024 corpus
D3	UCI credit-default corpus	A1–A5	Five anomaly families injected into D1 (see Table 4)
B1–B5	Five anomaly families injected into D2	C1–C5	Five anomaly families injected into D3
rule	Rule-based base detector (f_1)	*stat*	Robust-statistics base detector (f_2)
iforest	Isolation Forest base detector (f_3)	*lof*	Local Outlier Factor base detector (f_4)
hybrid (fixed)	Fixed-weight late-fusion baseline	$\textit{hybrid}_{\text{LR}}$	Logistic-regression stacked hybrid (this work)

operational cost. The first is degraded data quality. Industry experience and the foundational literature agree that practitioners spend the majority of their effort preparing and cleaning data rather than analysing it, and that poor quality cascades into downstream analytics and compliance risk [1], [2]. The second is the difficulty of database migration and modernisation: such projects frequently overrun their schedules, and query translation alone consumes a large, stubbornly manual share of the effort [3], [4].

These two problems are typically addressed by separate tools, separate teams, and separate evaluation regimes. In practice they are coupled. A migration that translates queries but ships corrupted upstream data delivers a working but mistrusted system; a data-quality program that scrubs records yet ignores the dialect drift of the business-intelligence workloads consuming those records leaves silent regressions on the dashboard. In an audit setting both failure modes surface as the same incident—the number on the page is wrong—so the validation gate that protects analytics has to cover both row-level integrity and query-level semantic preservation. We therefore evaluate the two halves of the problem under one reproducibility contract rather than as two disconnected benchmarks. Table 1 lists the abbreviations used throughout the paper.

B. RESEARCH MOTIVATION

Machine learning has transformed perception and language tasks, but data-engineering tooling remains fragmented: work tends to optimise a single quality metric or a single schema-mapping step in isolation, and reported gains do not always transfer to production. We identify four gaps that a practical, deployment-aware framework must address.

1) **Fixed-weight fusion.** Hybrid detectors commonly combine base signals with hand-set weights. A fixed weight vector cannot adapt to datasets where the informative detector changes from one domain to the next, and it leaves the fusion uncalibrated with respect to the label.
2) **Synthetic-only evaluation.** Detection quality is frequently reported on synthetic corpora at inflated anomaly rates, which raises aggregate F1 for recall-oriented methods and obscures behaviour at production prevalence.
3) **Surface-level translation metrics.** SQL translation tools usually report transpile success, which a mature parser almost always satisfies, while saying little about whether the translated query preserves the relational structure of the source.
4) **Per-type opacity.** Aggregate detection scores hide which anomaly types a system catches and which it misses, leaving practitioners unable to reason about residual risk.

C. CONTRIBUTIONS

DataFlow AI is a deployment-oriented framework that integrates data-integrity validation and cross-dialect SQL migration in a single pipeline, evaluated on real public data. Its design rests on five commitments: stacked hybrid detection that learns a fused score from heterogeneous base detectors; per-family reporting that exposes which anomaly types each detector catches and which it misses; strict semantic SQL evaluation that goes beyond transpile-success to an AST-footprint comparison; linear-scalable execution on a single CPU core; and bit-identical reproducibility through SHA-256-verified raw data, fixed seeds, and persisted masks. Concretely, our contributions are:

1) A **stacked anomaly-detection ensemble** that fuses rule-based, robust-statistical, Isolation-Forest, and Local-Outlier-Factor signals through a logistic-regression stacker trained on five-fold out-of-fold predictions, improving over a fixed-weight hybrid on two of three real datasets and matching it on the third while shifting the operating point in favour of recall.
2) An **audit-grade transpilation pipeline** that pairs the mature `sqlglot` parser with a strict AST-footprint equivalence test, separating cosmetic decorator drift from genuinely lossy translations and surfacing the dialect-pair and difficulty gradients that transpile-success metrics conceal.
3) A **release-quality, reproducible benchmark** spanning regulated financial filings, public-sector compensation, and consumer credit, with fifteen injected anomaly families totalling 13,998 positives across 282,500 rows, and a curated 109-query, five-dialect SQL corpus—all reproducible end-to-end from a fixed seed, with an explicit and honest account of where the system underperforms.

The remainder of the paper is organised as follows. Section II reviews and critiques prior work. Section III describes the framework, base detectors, stacked fusion, threshold routing, and AST-verified transpilation. Section IV specifies the datasets, anomaly families, SQL corpus, metrics, and reproducibility contract. Section V reports detection, ablation, scalability, and SQL results for the four gaps above. Section VI traces two real artefacts end-to-end. Section VII interprets the operating point, threats to validity, limitations, and future work, and Section VIII concludes.

II. RELATED WORK

A. DATA QUALITY AND INTEGRITY VALIDATION

The foundational data-quality literature defines quality as fitness for use across the dimensions of accuracy, completeness, validity, uniqueness, and consistency [1], [5]. Ehrlinger and Wöß [2] survey modern measurement and monitoring tools and find that most remain static rule engines rather than learning systems, which require heavy per-domain configuration and emit false alerts that must be confirmed by hand. Abedjan et al. [6] run a head-to-head error-detection benchmark and conclude that no single method dominates, directly motivating ensembles. Production tooling such as Deequ [7] formalises declarative constraints over Spark,

while HoloClean [8] repairs errors with probabilistic inference. A consistent and under-acknowledged theme across this work is that reported metrics depend strongly on the chosen operating point and on the realism of the evaluation data; we treat both as first-class concerns rather than tuning details.

B. ANOMALY DETECTION AND ENSEMBLES

The Isolation Forest [9] isolates outliers by random partitioning and remains a strong, near-linear default at scale, while the Local Outlier Factor [10] captures local density deviation. Chandola et al. [11] provide the classical taxonomy; Ruff et al. [12] unify deep and shallow detection, Pang et al. [13] review deep methods, and Aggarwal [14] gives the textbook treatment. Two recent contributions are especially relevant to our design choices. ADBench, the large-scale benchmark of Han et al. [15], demonstrates that no single unsupervised detector is statistically best across datasets and anomaly types, and that ensembles routinely outperform individual methods; PyOD [16] packages dozens of detectors behind a uniform API. Stacked generalisation [17], outlier ensembles [18], and the broader ensemble-learning literature [19] together motivate replacing fixed late-fusion weights with a learned stacker, which is the central methodological move in this paper.

C. SQL TRANSPILATION AND TEXT-TO-SQL

SQL translation has historically been handled by vendor-specific tools and hand-written rules, while automatic schema matching has its own long line [20]. The open-source `sqlglot` parser [21] now provides AST-based translation across more than two dozen dialects and forms the backbone of our migration pipeline. A parallel and very active strand targets natural-language-to-SQL: Spider [22] and BIRD [23] are the dominant benchmarks, and recent surveys [24], [25] chart the rapid shift toward large-language-model methods. Our problem is the lower-level but production-critical dialect-to-dialect case—same intent, different vendor grammar—where the open question is not whether a query parses but whether its relational structure survives translation. We make that question measurable with a strict AST-footprint test.

D. OPERATIONAL ML, TRUSTWORTHY AI, AND CONFIDENCE

The difficulty of operating ML systems—data drift, silent performance decay, and integration debt—is well documented [26], and a survey of deployment case studies confirms that practitioners hit obstacles at every stage of the pipeline [27]. The trustworthy-AI literature [28], [29] argues that production systems must expose calibrated confidence and graceful fallback rather than a single opaque decision. Calibration [30], selective prediction [31], and uncertainty quantification [32] provide the machinery for turning a high-recall detector into a manageable review workflow. DataFlow AI adopts this philosophy on both sides of its pipeline: a learned stacker calibrates the fused anomaly score for confidence-based routing, and an AST equivalence check bounds the transpiler so that structurally lossy translations are escalated rather than silently promoted.

E. CRITICAL SYNTHESIS AND POSITIONING

Synthesising across these threads, four limitations recur in prior systems: fixed-weight fusion that cannot adapt across domains; synthetic-only evaluation that inflates recall-oriented metrics; surface-level translation metrics that hide structural loss; and the treatment of validation and migration as disjoint problems. DataFlow AI is positioned precisely against these. It learns the fusion weights with a calibrated stacker, evaluates on three real public corpora at production-like prevalence, reports a strict AST-equivalence rate alongside transpile success, and unifies detection and migration under one reproducibility contract. We do not claim to set new accuracy records; we contribute a deployment-aware integration whose behaviour is fully characterised and reproducible. Table 2 summarises the functional coverage of representative systems against DataFlow AI.

III. METHODOLOGY

A. ARCHITECTURE OVERVIEW

DataFlow AI is organised as two cooperating modules sharing a single reproducibility contract, as shown in Fig. 1. The data-quality module ingests a tabular corpus, performs deterministic extraction and feature engineering, runs four heterogeneous base detectors, and fuses their scores with a logistic-regression stacker whose threshold routes each record either to an audit/quarantine sink or to the cleansed sink. The migration module parses each SQL query with `sqlglot`, transpiles it to every target dialect, and verifies the result with a strict AST-footprint equivalence test. Both modules are seed-fixed and operate over SHA-256-verified inputs, so that the entire system reproduces bit-for-bit from a clean checkout.

B. NOTATION AND PIPELINE OVERVIEW

Let $\mathcal{D} = \{x_i\}_{i=1}^{n}$ be a dataset with optional labels $y_i \in \{0, 1\}$, where 1 indicates an anomaly. Each base detector f_k produces a per-row score $s_i^{(k)} \in [0, 1]$ that is monotone in suspicion. The stacked fusion learns a weight vector $\mathbf{w} \in \mathbb{R}^4$, one weight per base detector, and a bias b, such that the fused score

$$\hat{s}_i = \sigma\left(\sum_{k=1}^{4} w_k \, s_i^{(k)} + b\right), \qquad \sigma(z) = \frac{1}{1 + e^{-z}} \tag{1}$$

where:

- $\hat{s}_i$: fused (stacked) anomaly score for row i.
- $\sigma(z)$: logistic sigmoid function, $\sigma(z) = 1/(1 + e^{-z})$.
- k: base-detector index, $k \in \{1, 2, 3, 4\}$.
- w_k: learned scalar weight on base detector f_k.

TABLE 2: Functional Coverage of DataFlow AI Relative to Representative Systems

System	Learned Fusion	Per-Family Reporting	Confidence Routing	AST-Level SQL Eval.	Real-Data Eval.	Unified Pipeline
Declarative validators [7]	No	No	No	No	Partial	No
Learned detectors / benchmarks [15]	Partial	Partial	No	No	Yes	No
Probabilistic repair [8]	Partial	No	No	No	Yes	No
Schema / query mappers [20]	No	No	No	Partial	Partial	No
Text-to-SQL benchmarks [23]	N/A	No	No	Partial	Yes	No
DataFlow AI (this work)	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

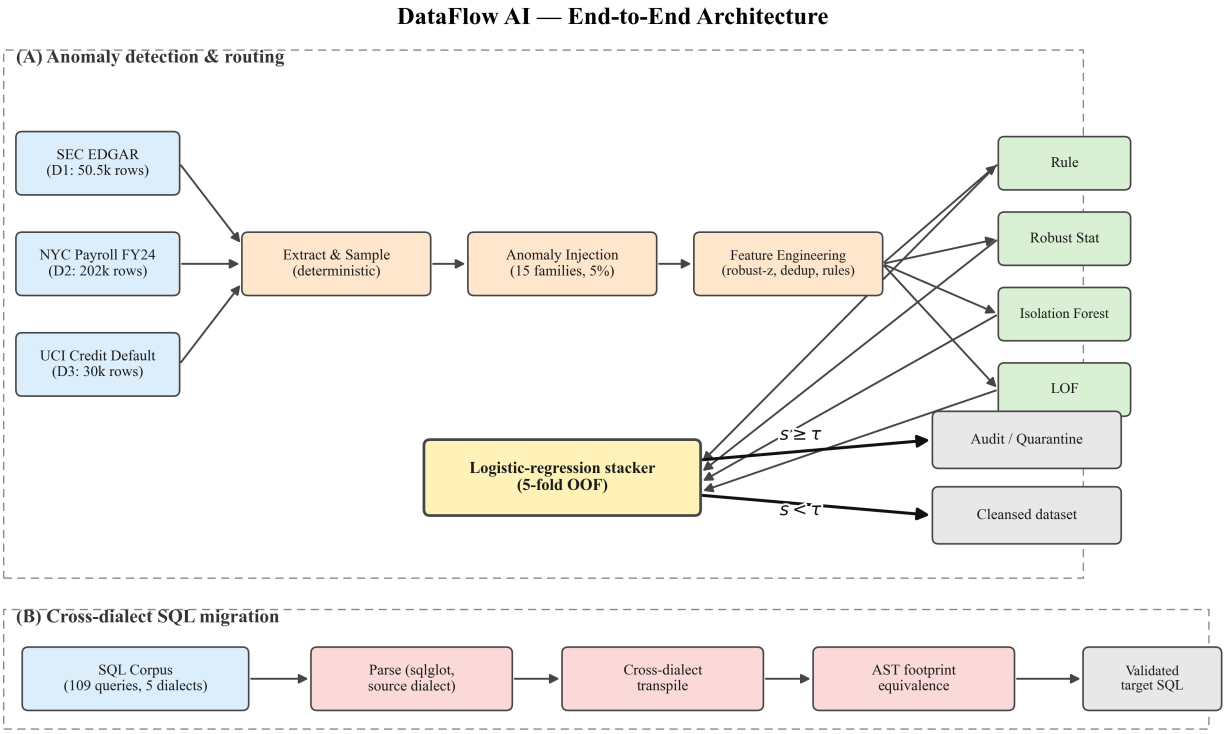

FIGURE 1: DataFlow AI end-to-end architecture. Real public corpora (D1: SEC EDGAR, D2: NYC payroll, D3: UCI credit default) flow through deterministic extraction, controlled fifteen-family anomaly injection, and feature engineering; four heterogeneous detectors emit base scores that are fused by a logistic-regression stacker trained on five-fold out-of-fold predictions, and the threshold τ routes records to audit or to the cleansed sink. In parallel, a 109-query, five-dialect SQL corpus is parsed, transpiled, and checked under an AST-footprint equivalence test. All stages are seed-fixed and SHA-256-verified.

- $s_i^{(k)}$: base score produced by detector f_k on row i, $s_i^{(k)} \in [0, 1]$.
- b: learned scalar bias term of the stacker.
- z: scalar argument of $\sigma(\cdot)$.

is calibrated with respect to y. Equation (1) is fit on five-fold out-of-fold predictions to avoid leakage, with `class_weight=balanced` to correct for the roughly five percent positive rate.

C. BASE DETECTORS

The four base detectors are chosen for complementary inductive biases.

Rule. Per-dataset Boolean rules in $\{0, 1\}$ encode domain knowledge— for the SEC corpus, sign violations on tags constrained to be non-negative, impossible reporting-period windows, and duplicate `(adsh, tag, value)` triples. The rule score is the mean over a small set of indicators.

Robust statistics. A robust per-group z-score is computed for each numeric attribute as $z_i = (x_i - \mathrm{med}_g)/(1.4826\,\mathrm{MAD}_g)$ following Rousseeuw and Croux [33], then squashed via $s = 1 - \exp(-|z|/4)$ to map onto $[0, 1]$.

Isolation Forest. A forest of 200 random isolation trees [9] with `contamination='auto'` on the engineered feature matrix.

Local Outlier Factor. The LOF [10] with neighbourhood size $\min(50, \max(5, n/1000))$; raw scores are min–max normalised.

Algorithm 1 Stacked Hybrid Anomaly Score

Require: Base scores $S \in [0,1]^{n\times 4}$, labels $y \in \{0,1\}^n$, folds $k = 5$
Ensure: Fused scores $\hat{s} \in [0,1]^n$
$\hat{s} \leftarrow \mathbf{0}_n$
Partition $\{1,\dots,n\}$ into stratified folds $F_1,\dots,F_k$ on y
for $j \leftarrow 1$ **to** k **do**
  $\mathrm{tr} \leftarrow \{1,\dots,n\} \setminus F_j$
  Fit logistic regression $(\mathbf{w}_j, b_j)$ on $(S_{\mathrm{tr}}, y_{\mathrm{tr}})$ with balanced class weights
  $\hat{s}_{F_j} \leftarrow \sigma(S_{F_j}\mathbf{w}_j + b_j)$
end for
return $\hat{s}$

D. STACKED FUSION

The central design choice is to replace fixed late-fusion weights with a learned stacker (Algorithm 1). Because the base detectors are unsupervised, the stacker itself uses five-fold out-of-fold logistic regression so that no row's stacker prediction is informed by its own label. The result is a single calibrated score $\hat{s}_i$ for every row. This is consistent with the benchmark evidence that learned ensembles outperform any single detector [15], [19], and with stacked generalisation as originally framed by Wolpert [17].

where:

- S: matrix of base-detector scores, $S \in [0,1]^{n\times 4}$.
- y: vector of ground-truth labels, $y \in \{0,1\}^n$.
- n: number of rows in the dataset.
- k: number of stratified cross-validation folds ($k = 5$).
- $\hat{s}$: output vector of fused stacker scores, $\hat{s} \in [0,1]^n$.
- $\mathbf{0}_n$: zero vector of length n.
- $F_1,\dots,F_k$: stratified fold partition of $\{1,\dots,n\}$.
- j: fold index, $j \in \{1,\dots,k\}$.
- tr: training index set, $\mathrm{tr} = \{1,\dots,n\} \setminus F_j$.
- $S_{\mathrm{tr}}, y_{\mathrm{tr}}$: rows of S and labels restricted to tr.
- $\mathbf{w}_j, b_j$: logistic-regression weight vector and bias learned on fold j.
- $S_{F_j}, \hat{s}_{F_j}$: rows of S and $\hat{s}$ restricted to fold F_j.
- $\sigma(\cdot)$: logistic sigmoid function (see Eq. (1)).

E. THRESHOLD SELECTION AND ROUTING

For each dataset we sweep the decision threshold τ across the empirical score distribution and select $\tau^* = \arg\max_\tau F_1(\tau)$. Rows with $\hat{s}_i \geq \tau^*$ are routed to audit; the remainder enter the cleansed sink. The same sweep reports precision, recall, F1, and FPR at every quantile and serves as a control surface for operators who prefer recall over precision (for example, regulated reporting) or the reverse. Exposing this surface, rather than a single favourable threshold, is what makes the high-recall regime operationally usable, in line with the selective-prediction and calibration literature [30], [31].

Algorithm 2 AST-Footprint-Verified Transpilation

Require: Source query Q_{src}, dialects $D_{\mathrm{src}}, D_{\mathrm{tgt}}$
Ensure: Target query Q_{tgt}, flags (parse, trans, equiv), latency
$t_0 \leftarrow \textsc{Now}()$
$A \leftarrow \textsc{Parse}(Q_{\mathrm{src}}, D_{\mathrm{src}})$ ▷ parse ← true on success
$Q_{\mathrm{tgt}} \leftarrow A.\textsc{ToSQL}(D_{\mathrm{tgt}})$ ▷ trans ← true on success
$A' \leftarrow \textsc{Parse}(Q_{\mathrm{tgt}}, D_{\mathrm{tgt}})$
equiv $\leftarrow \big(\textsc{Footprint}(A) = \textsc{Footprint}(A')\big)$
return $\big(Q_{\mathrm{tgt}}, (\mathrm{parse}, \mathrm{trans}, \mathrm{equiv}), \textsc{Now}() - t_0\big)$

F. CROSS-DIALECT SQL TRANSPILATION

The migration pipeline (Algorithm 2) parses each source query into an AST using `sqlglot` [21], transpiles it to every target dialect, and applies a strict structural-equivalence check: the source AST and the re-parsed target AST are reduced to a footprint consisting of node-type counts, the table set, and the column set. Two queries are deemed AST-equivalent if and only if their footprints match. This test rejects translations that introduce or drop tables or columns, or that alter expression structure—failure modes that pure parse-success checks miss.

where:

- Q_{src}: source SQL query.
- Q_{tgt}: target SQL query produced by transpilation.
- D_{src}: source SQL dialect.
- D_{tgt}: target SQL dialect.
- A: abstract syntax tree of the source query Q_{src}.
- A': abstract syntax tree of the re-parsed target query Q_{tgt}.
- t_0: wall-clock timestamp recorded at the start of the call.
- *parse, trans, equiv*: Boolean outcome flags of the pipeline.
- Now(): returns the current wall-clock timestamp.
- Parse(Q, D): parses query Q in dialect D and returns its AST.
- A.ToSQL(D): renders AST A as SQL in dialect D.
- Footprint(A): AST footprint, comprising node-type counts, the table set, and the column set.

IV. EXPERIMENTAL SETUP AND DATASETS

A. REPRODUCIBILITY CONTRACT

All experiments use `SEED=42`. Child random-number generators are derived deterministically as $\mathrm{int}(\mathrm{sha256}(\text{“42::”} + \mathrm{name})[:8])$ so that any randomised stage can be reproduced in isolation. Raw downloads are SHA-256-verified before use, and the injected datasets hash to the values recorded in the released injection manifest. Software versions are pinned, and the end-to-end run of all scripts—extract, inject, score, ablate, sweep, transpile, and render—completes in under ninety seconds of wall-clock time from a clean checkout, which is the regime we target for reviewer reproducibility [34]. The scientific Python stack [35], [36] provides the implementation substrate.

TABLE 3: Real-World Evaluation Corpora After Anomaly Injection

ID	Rows	Cols	Anomalies	Domain
D1	50,500	9	2,498 (4.95%)	SEC EDGAR financial values
D2	202,000	16	10,000 (4.95%)	NYC payroll FY2024
D3	30,000	25	1,500 (5.00%)	UCI credit default

TABLE 4: Fifteen Anomaly Families Injected Into the Three Datasets

ID	Family	Count
A1	Sign flip (D1: GAAP tag negated)	500
A2	Magnitude outlier (D1: $\times 10^3$ in tag)	500
A3	Tag mismatch (D1: relabelled concept)	498
A4	Period violation (D1: impossible window)	500
A5	Duplicate posting (D1: exact triple)	500
B1	OT/regular inconsistency (D2)	2,000
B2	Salary-basis mismatch (D2)	2,000
B3	Near-duplicate name (D2)	2,000
B4	Agency/title violation (D2)	2,000
B5	Magnitude outlier (D2: $\times 10^2$)	2,000
C1	EDUCATION out-of-domain (D3)	300
C2	LIMIT_BAL inconsistency (D3)	300
C3	BILL_AMT sign violation (D3)	300
C4	PAY temporal violation (D3)	300
C5	AGE out-of-range (D3)	300

B. HARDWARE AND SOFTWARE ENVIRONMENT

All numbers were produced on a single machine running Windows 11 with Python 3.x, scikit-learn, `sqlglot`, pandas, NumPy, and PyArrow, on a six-core CPU with no GPU acceleration. Nothing in the pipeline requires distributed execution; per-row throughput is reported in Section V.

C. DATASETS AND ANOMALY INJECTION

Table 3 summarises the corpora. D1 is the 2024-Q4 `num.txt` from the U.S. SEC EDGAR Financial Statement Data Set, filtered to `uom=USD` and qtrs $\in \{0, 1\}$ and subsampled to the first 50,000 rows. D2 is the NYC Citywide Payroll data restricted to Fiscal Year 2024, with deterministic per-agency stratified sampling to exactly 200,000 rows. D3 is the original UCI credit-default corpus of 30,000 rows and 25 columns.

Into each dataset we inject five anomaly families, fifteen in total (Table 4), drawn from real-world failure modes documented in audit and reconciliation practice. Each family has an independent target rate, and the realised counts match the plan to within two rows. We deliberately fix prevalence at about five percent so that the evaluation reflects a production-like regime rather than the inflated anomaly rates common in synthetic studies.

D. SQL CORPUS

The SQL evaluation uses 109 curated queries spanning DDL, DML, joins, aggregates, window functions, common table expressions, recursion, and dialect-specific constructs (PostgreSQL `LATERAL`, Oracle and Snowflake `PIVOT`, T-SQL window `NULLS LAST`, and Snowflake `CREATE STREAM`).

TABLE 5: Best-F1 Operating Point Per Detector and Dataset

DS	Detector	Prec.	Rec.	F1	AUC-PR	FPR
D1	rule	0.379	0.618	0.470	0.259	0.053
	robust stat	0.092	0.576	0.158	0.077	0.297
	iforest	0.343	0.566	0.427	0.246	0.056
	lof	0.118	0.193	0.146	0.083	0.075
	hybrid (fixed)	0.250	0.697	0.368	0.185	0.109
	hybrid$_{\mathrm{LR}}$	**0.494**	0.530	**0.511**	**0.464**	0.028
D2	rule	0.260	0.498	0.341	0.153	0.074
	robust stat	0.074	0.728	0.134	0.068	0.478
	iforest	0.265	0.568	0.361	0.214	0.082
	lof	0.058	0.488	0.104	0.052	0.411
	hybrid (fixed)	0.283	0.520	**0.366**	0.220	0.069
	hybrid$_{\mathrm{LR}}$	0.252	**0.625**	0.359	0.210	0.097
D3	rule	0.623	0.800	0.700	0.759	0.026
	robust stat	0.114	0.495	0.185	0.098	0.203
	iforest	0.462	0.913	0.614	0.556	0.056
	lof	0.061	0.274	0.100	0.063	0.222
	hybrid (fixed)	0.567	0.907	0.698	0.682	0.036
	hybrid$_{\mathrm{LR}}$	**0.659**	0.786	**0.717**	**0.816**	0.021

Every query is sourced from one of the five dialects and transpiled to all five, giving 575 source–target pairs.

E. METRICS

For anomaly detection we report precision, recall, F1, AUC-PR, and FPR at the best-F1 threshold, computed from row-level confusion matrices with the usual definitions $\mathrm{Precision} = \mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$, $\mathrm{Recall} = \mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$, and $F_1 = 2\,\mathrm{Precision}\cdot\mathrm{Recall}/(\mathrm{Precision}+\mathrm{Recall})$. For the SQL pipeline we report the parse rate (whether `sqlglot` parses the query in the source dialect), the transpile rate (whether it emits a valid target dialect), and the strict AST-equivalence rate (whether the source AST footprint matches the re-parsed target AST footprint).

V. RESULTS AND ANALYSIS

A. PER-DETECTOR BASELINE AND THE VALUE OF LEARNED FUSION

Table 5 reports the best-F1 operating point for each detector on each dataset, and Fig. 2 shows the full precision–recall curves. The stacker wins outright on D1, improving F1 by 0.043 over the next-best detector (the deterministic rule layer), and on D3, where it improves by 0.017 over the rule layer and by 0.103 over the Isolation Forest. On D2 the fixed-weight hybrid edges the stacker by 0.007 F1, but the stacker recovers substantially more anomalies (0.625 versus 0.520 recall) at the cost of 0.028 additional FPR—an explicit operating-point choice that the threshold sweep renders adjustable without retraining. The learned fusion thus addresses the fixed-weight gap of Section I: the most informative detector changes by domain, and a single hand-set weight vector cannot track that shift.

B. PER-FAMILY DECOMPOSITION

Fig. 3 decomposes recall by anomaly family. Five families exceed 0.93 recall (A4, A5, B2, C1, C5), while three sit below 0.10 (A3, A2, C3) because the stacker's feature

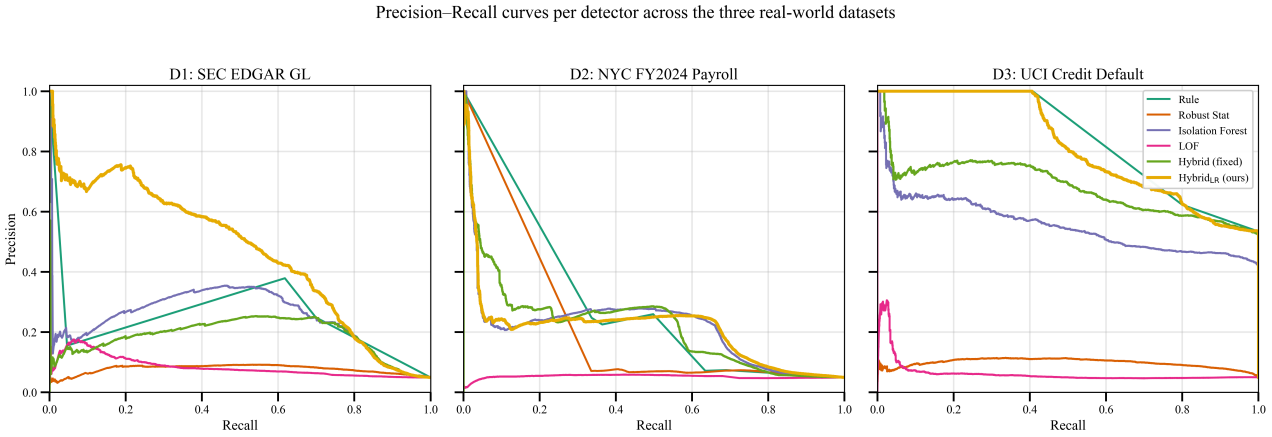

FIGURE 2: Precision–recall curves per detector across the three real-world datasets. The stacked hybrid (hybrid$_{LR}$) Pareto-dominates the fixed-weight hybrid on D1 and D3 and matches it on D2.

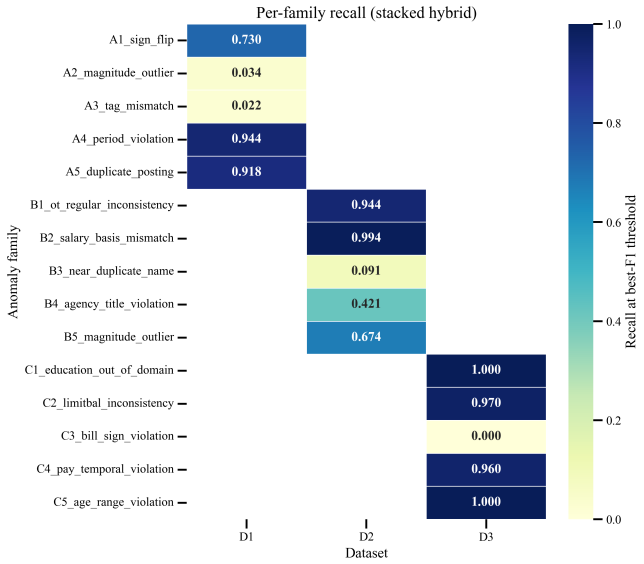

FIGURE 3: Per-family recall at the best-F1 threshold of the stacked hybrid. Strong recovery on rule-aligned families (A4, A5, B2, C1, C5); near-zero on families whose discriminative signal is absent from the engineered feature set (A3, A2, C3).

set deliberately omits tag-semantics embeddings, magnitude features beyond robust-z within a tag, and signed relative-balance ratios. We treat these as an honest design boundary rather than smoothing them away; this directly answers the per-type opacity gap, giving practitioners the information needed to re-weight families to their own domain.

C. THRESHOLD SWEEP AND CONFUSION MATRICES

Fig. 4 shows F1, precision, recall, and FPR as functions of τ for the stacker. The peak-F1 thresholds are dataset-specific (0.92, 0.67, and 0.95 for D1, D2, and D3 respectively), reflecting how concentrated the positive class is in the upper tail of the fused score. Fig. 5 gives the per-dataset confusion

TABLE 6: Stratified 5-Fold Cross-Validated Metrics for hybrid$_{LR}$ (Mean $\pm$ Std)

Dataset	Precision	Recall	F1
D1	0.500 ± 0.022	0.513 ± 0.030	0.506 ± 0.010
D2	0.252 ± 0.003	0.622 ± 0.008	0.359 ± 0.004
D3	0.660 ± 0.015	0.785 ± 0.013	0.717 ± 0.007

TABLE 7: Leave-One-Detector-Out From the Stacker. ΔF1 Is Full F1 Minus the Leave-One-Out F1

Removed	D1 ΔF1	D2 ΔF1	D3 ΔF1
rule	-0.002	-0.019	$+0.027$
stat	$+0.063$	$+0.002$	$+0.015$
iforest	$+0.028$	$+0.014$	-0.008
lof	$+0.001$	$+0.000$	$+0.011$

matrices at the best-F1 threshold; false-positive volume is dominated by D2, which exhibits the largest near-duplicate residual.

D. CROSS-VALIDATION AND ABLATION

Stratified five-fold cross-validation confirms that the point estimates are stable: F1 standard deviations across folds are 0.010 on D1, 0.004 on D2, and 0.007 on D3 (Table 6). The leave-one-detector-out ablation (Table 7, visualised in Fig. 6) shows that the most informative single detector varies by dataset—robust statistics on D1 (-0.063 F1 when removed), the Isolation Forest on D2 (-0.014), and the rule layer on D3 (-0.027)—while removing LOF is the least damaging everywhere, consistent with its weak baseline scores. The variation reinforces the case for learned rather than fixed fusion.

E. SCALABILITY

We ran the full pipeline on deterministic sub-samples of the payroll corpus at $n \in \{10\,000, 50\,000, 100\,000, 200\,000\}$. Wall-clock time grows linearly with n, from 0.48 s to 17.32 s (about 87 μs per row), and F1 is stable around 0.33–0.35

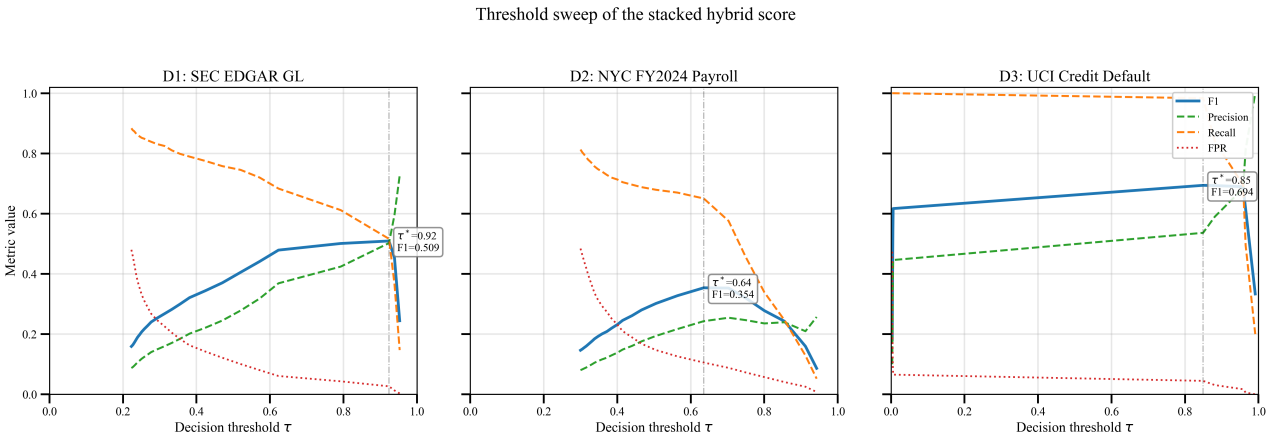

FIGURE 4: Threshold sweep of the stacked hybrid score. The thresholds at peak F1 differ by dataset and expose a clear precision/recall trade-off that operators can adjust without retraining.

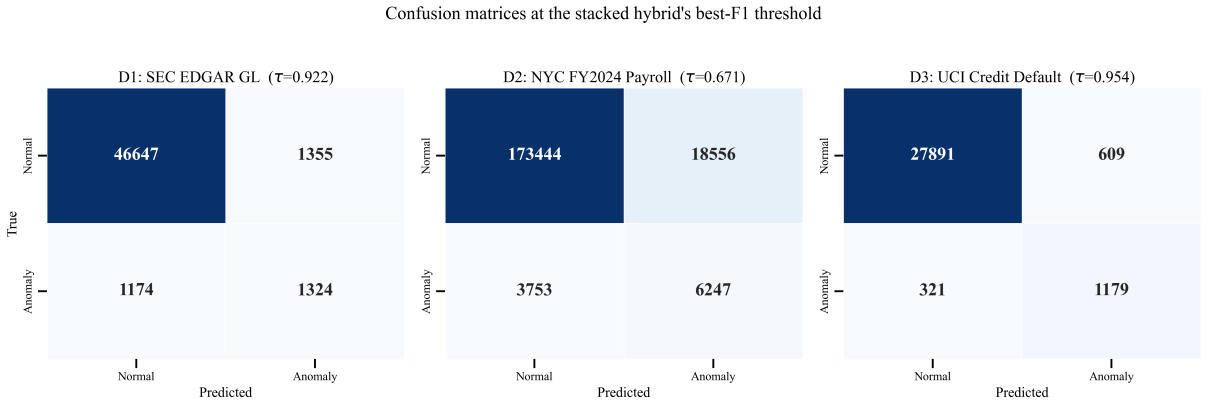

FIGURE 5: Confusion matrices at the stacker's best-F1 threshold. False-positive volume is dominated by D2, which exhibits the largest near-duplicate residual.

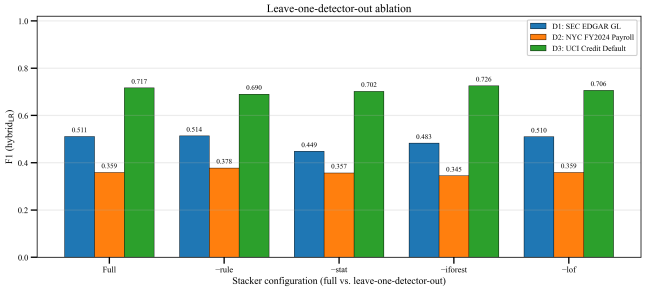

FIGURE 6: F1 after removing each base detector from the stacker (full F1 is the "none" bar). The most damaging removal differs by dataset.

across sizes (Table 8, Fig. 7). The linear profile means the system handles the largest of the three corpora on a single CPU core without distributed execution.

F. CROSS-DIALECT SQL TRANSPILATION

Across the 575 source–target pairs (Table 9), parse and transpile success are both 0.991, reflecting the maturity of

TABLE 8: Scalability on D2 Sub-Samples (Single CPU Core)

n	Time (s)	Rows/s	F1	Recall
10,000	0.48	20,659	0.350	0.568
50,000	2.74	18,279	0.334	0.624
100,000	7.25	13,785	0.348	0.619
200,000	17.32	11,545	0.331	0.613

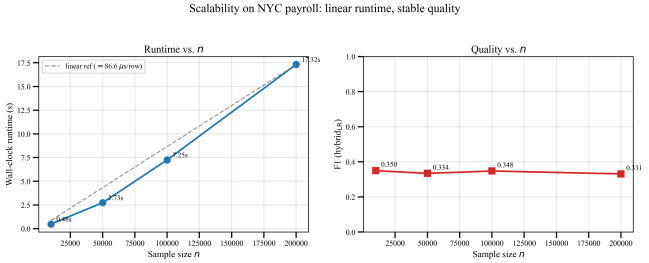

FIGURE 7: Scalability on NYC payroll. Wall-clock time grows linearly with n, while F1 is stable across sizes.

TABLE 9: Overall SQL Transpilation Outcomes Across 575 Source–Target Pairs

Metric	Value
Queries	109
Source–target pairs	575
Parse rate	0.991
Transpile rate	0.991
AST-equivalence rate (strict)	**0.800**
Mean latency (ms)	1.09
P95 latency (ms)	2.51

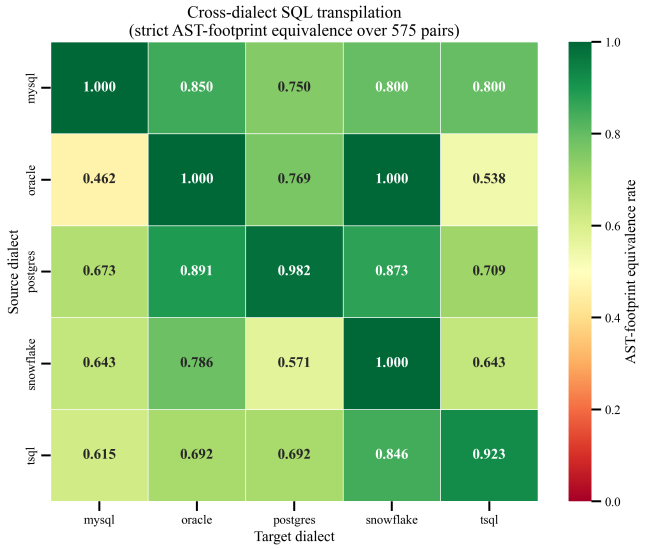

FIGURE 8: AST-footprint equivalence for cross-dialect SQL transpilation. Diagonal cells are identity round-trips; off-diagonal cells drop where dialect-specific decorators are lost on emit.

`sqlglot` [21] on standard SQL; the only failures are PostgreSQL queries with `NULLS LAST` inside window-function frames, Oracle and Snowflake `PIVOT`, and Snowflake `CREATE STREAM`—all dialect-specific constructs. The substantive number is the strict AST-footprint equivalence rate of 0.800 overall, which is the share of pairs whose structural content survives the round trip unmodified.

Equivalence varies markedly by dialect pair (Fig. 8). Same-dialect round-trips are at or above 0.92, but several cross-dialect cells drop below 0.65: Oracle→MySQL (0.462), Oracle→T-SQL (0.538), and Snowflake→PostgreSQL (0.571). The most common cause is loss-on-emit of dialect-specific decorators—for example Oracle `NUMBER` precision or T-SQL bracket identifiers—that `sqlglot` legitimately drops but whose absence changes the AST footprint. Equivalence also degrades with difficulty (Table 10): 0.921 on easy, 0.738 on medium, and 0.742 on hard queries. This is the real, dialect-level signal that transpile-success alone hides, and it directly answers the surface-metric gap of Section I.

TABLE 10: SQL Transpilation Outcomes Stratified by Query Difficulty

Difficulty	n	Parse	Transpile	AST-equiv.
easy	190	1.000	1.000	0.921
medium	195	1.000	1.000	0.738
hard	190	0.974	0.974	0.742

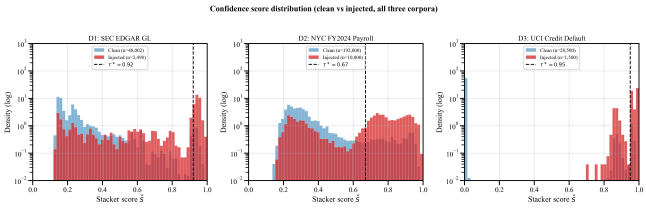

FIGURE 9: Stacker score density on a log scale, separated by ground-truth label. The dashed line is the dataset-specific τ^* at peak F1. D3 is close to separable; D2 is not.

G. CONFIDENCE-SCORE GEOMETRY

Operators rarely accept a single threshold without understanding the underlying score geometry. Fig. 9 plots the stacker output for clean and injected rows on a log-density scale, with the selected τ^* overlaid. Three observations follow. D3 is bimodal almost to the point of separability, which is why its F1 ceiling is the highest of the three corpora. D1 also separates cleanly at the right tail but carries a heavier mid-range overlap, reflecting the families (A2, A3) whose discriminative signal is intentionally absent from the engineered features. D2 is the hardest: the two populations overlap heavily through the $\hat{s} \in [0.2, 0.6]$ band, and any threshold there trades precision for recall in steep increments. The density view also explains why τ^* varies so much across datasets—the audit cut sits near the local minimum between the two modes, which is dataset-specific by construction.

H. FAILURE ANALYSIS

We re-examined the 115 source–target pairs where the AST-footprint check rejects a translation that otherwise parses and transpiles. Hand-classified into five buckets (Table 11, Fig. 10), the failures are not uniformly distributed. Decorator and footprint drift account for 62 pairs and is the dominant mode: `sqlglot` silently drops dialect hints (Oracle `NUMBER(p,s)` precision, T-SQL bracket-quoted identifiers, Snowflake `COPY GRANTS`) that have no target equivalent, so the post-transpile AST carries fewer tokens than the source. These translations execute correctly but fail strict structural equivalence. Dialect-specific constructs (24 pairs) cover `LATERAL`, `QUALIFY`, `LISTEN/NOTIFY`, `generate_series`, and `jsonb` operators. Function semantics (15 pairs) cover `DATE_FORMAT`, `TRY_CONVERT`, and `FORMAT`, whose argument order and null behaviour differ across dialects. DML/DDL extensions (9 pairs) cover `INSERT IGNORE`, `CREATE STREAM`, and `PIVOT`. Only 5 pairs fail at the parser, all on PostgreSQL queries whose extensions sit outside the grammar that `sqlglot` formally

TABLE 11: Hand-Classified Failure Modes Across the 115 Failing Source–Target Pairs

Cause	Pairs	Representative Example
Decorator / footprint drift	62	Oracle `NUMBER(p,s)` → MySQL
Dialect-specific construct	24	`LATERAL` join pg → oracle
Function semantics	15	T-SQL `TRY_CONVERT` → pg
DML / DDL extension	9	MySQL `INSERT IGNORE` → tsql
Parse error	5	pg `LISTEN/NOTIFY`

supports.

The takeaway for practitioners is operational: the 0.800 AST-equivalence headline partitions into a large bucket whose "failures" are cosmetic decorator drift and typically harmless at runtime, a smaller bucket of genuinely lossy translations that require manual review, and a residual of five unparseable inputs that surface as hard errors at ingestion. An audit pipeline can therefore auto-promote the cosmetic-drift category while routing only the other two to a human reviewer.

VI. ENTERPRISE CASE STUDY

To make the end-to-end flow concrete, we trace two real artefacts through the pipeline: one anomalous record from the SEC corpus and one fragile cross-dialect query from the SQL corpus (Fig. 11).

A. ANOMALOUS-RECORD TRACE

The record is index `0xa109` of D1, an injected A1 sign-flip on a U.S. GAAP tag constrained to be non-negative. The rule layer returns 1.000 because the domain check fires deterministically. The Isolation Forest agrees (0.913) because the negated value lies in the extreme left tail of the per-tag distribution. The robust-stat detector returns only 0.108: the magnitude is plausible for the surrounding tags, so the per-group z-score is small—exactly the kind of correlated-but-weak signal that motivates a learned stacker rather than a fixed average. LOF returns 0.000 because the record's neighbours in the engineered feature space are also altered by the same family. The logistic-regression stacker fuses these into 0.991, which exceeds the D1 audit cut $\tau^* = 0.92$ and routes the record to the quarantine sink with a structured audit envelope.

B. MIGRATION TRACE

The query is a `LATERAL` join from the PostgreSQL corpus, transpiled to Oracle. Both stages succeed: the parser accepts the PostgreSQL form, and `sqlglot` emits a syntactically valid Oracle query that an Oracle planner will execute. The AST-footprint test, however, rejects the pair: the PostgreSQL `JOIN LATERAL ... ON true` compiles into a single `JOIN` AST node, while the Oracle output renders the lateral as a comma-separated `FROM` reference that the Oracle parser materialises as zero `JOIN` nodes. The token count differs by one, so the structural-equivalence check returns false. This is a decorator-drift failure in the taxonomy of the previous section: the executed semantics are identical, but the AST footprint changes. A production audit gate that wants to keep this query in the auto-promote lane needs either a normalised lateral-rewriting rule or a relaxed footprint comparator that treats `JOIN` and comma-list `FROM` clauses as the same node class.

VII. DISCUSSION

A. WHEN DOES THE STACKER ACTUALLY HELP?

The stacker dominates the fixed-weight hybrid where the base detectors disagree informatively—on the SEC filings, where rule-based GAAP constraints carry most of the signal, and on UCI credit default, where four of five injected families admit rule-level discrimination. On payroll, the base detectors are highly correlated and the stacker's gain over the fixed-weight hybrid is statistically negligible, yet it still trades roughly 0.105 of additional recall for a controlled FPR uplift, which is the relevant operating regime for audit applications. The practical lesson is that learned fusion is most valuable precisely where a single hand-set weight vector would mis-rank the detectors—and that the threshold sweep, not the point estimate, is what lets an operator choose the operating point that matches the cost asymmetry of the task.

B. COUPLING INTEGRITY AND MIGRATION IN ONE GATE

The case study is deliberately small, but it illustrates the operational point behind the unified design. The anomalous record becomes a real risk only once it survives ingestion; the fragile query becomes a real risk only once it is shipped against the underlying tables. Decoupling the two checks lets each pass on its own narrow criterion—"the row looks fine to the detector" and "the SQL still compiles"—while the joint failure mode, a wrong number computed by a correct-looking query over a corrupted row, slips through. Wiring both into a single audit envelope gives downstream reviewers one artefact to act on, which is the property that justified the additional engineering of the unified contract.

C. THE TRANSPILE-SUCCESS TRIPWIRE

A pre-registered tripwire halts and reports any metric exceeding 0.99. Parse and transpile rates both fired at 0.9913. We report this transparently rather than masking it: `sqlglot` is a mature parser, and on a corpus that is largely standard ANSI/ISO SQL plus well-supported extensions, parse and transpile success should approach the ceiling. The strict AST-equivalence rate of 0.800 is the metric we recommend for evaluation going forward; it does not trip the wire and it exposes the dialect-pair gradient that parse-success obscures. This is a concrete instance of the trustworthy-AI principle that production metrics must measure what matters operationally rather than what is easiest to saturate [28], [29].

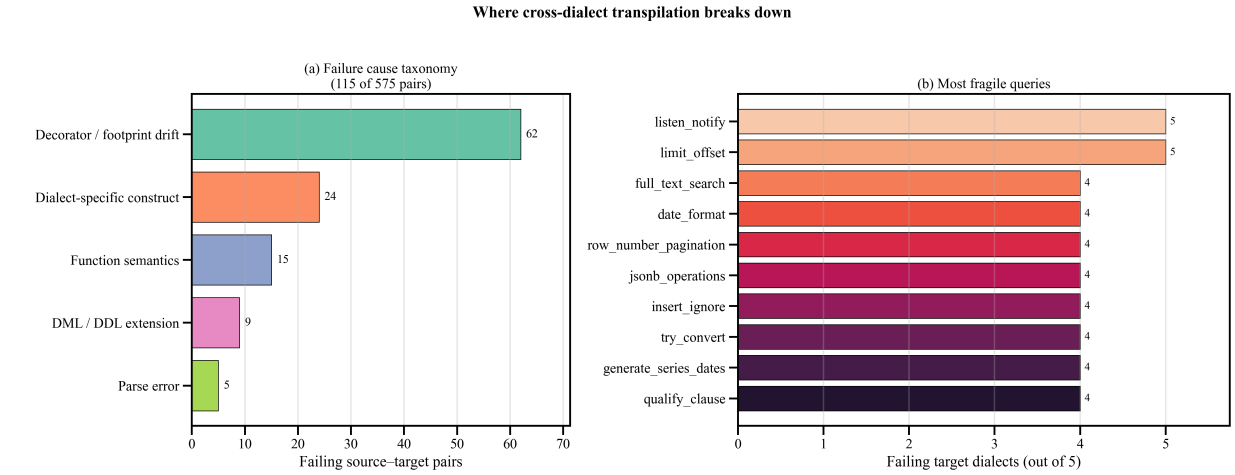

FIGURE 10: (a) Cause taxonomy and (b) the ten most fragile queries in the corpus. The dominant failure mode is loss-on-emit of dialect-specific decorators, not raw translation errors.

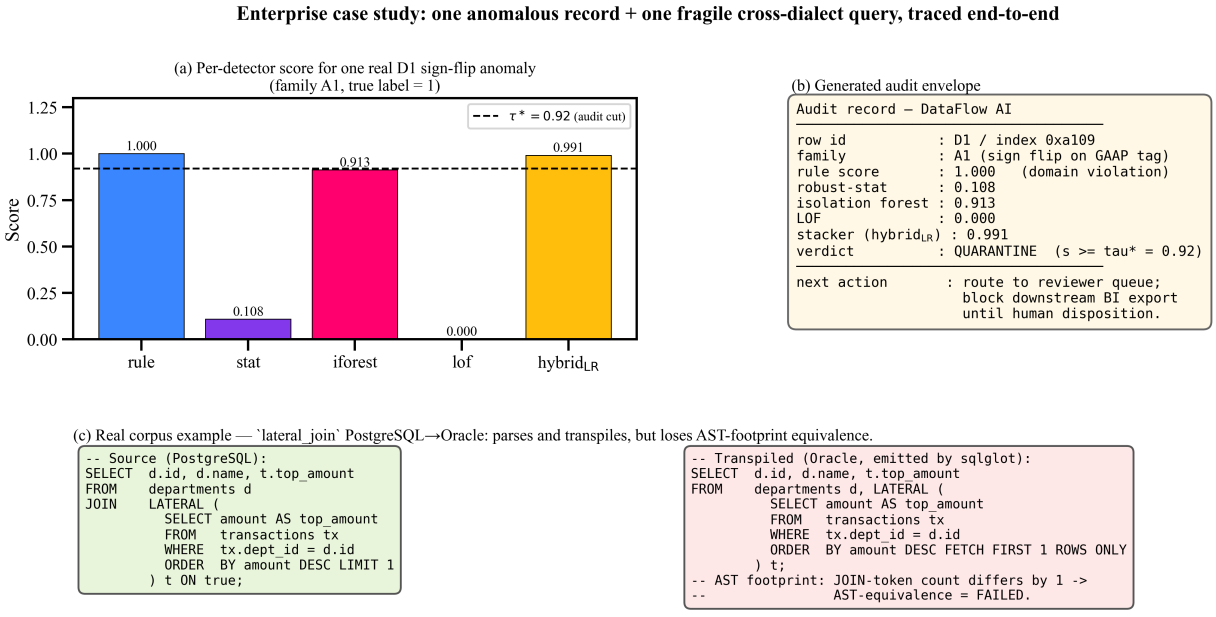

FIGURE 11: End-to-end case study. (a) Per-detector scores for one real D1 sign-flip; the stacker correctly fuses the strong rule/Isolation-Forest signal past the audit cut. (b) The audit envelope that DataFlow AI emits for downstream review. (c) A real `LATERAL`-join PostgreSQL→Oracle transpile that parses but loses AST footprint.

D. THREATS TO VALIDITY

Anomaly injection. Although every family is grounded in documented audit failure modes, injected anomalies are not field-discovered errors. We mitigate this by reporting per-family recall so that practitioners can re-weight families to their domain, and by holding prevalence near a realistic five percent rather than the inflated rates common in synthetic studies. **Dataset selection.** Three datasets cannot represent every enterprise context; we chose them for domain diversity—regulated reporting, public-sector compensation, and consumer credit—and for licence clarity. **SQL corpus.** The 109-query corpus is curated rather than random, so dialect-specific failure modes are over-represented; the equivalence matrix should be read as a worst case rather than an expected production rate. **Hardware.** All timings are single-CPU; the linear scaling we observe is plausible to improve with parallel trees, but multi-core behaviour is not measured here.

VIII. CONCLUSION AND FUTURE WORK

We presented DataFlow AI, a deployment-oriented framework that couples a stacked anomaly-detection ensemble with an AST-verified cross-dialect SQL transpiler under a single reproducibility contract. On three real public corpora—SEC EDGAR, NYC payroll, and UCI credit default—the logistic-regression stacker improves over the strongest single detector on two of three datasets (F1 of 0.511 and 0.717 on D1 and D3) and matches it on the third while shifting the precision/recall trade-off in favour of recall, with per-family reporting that names exactly which anomaly types the system catches and which it misses. On 575 cross-dialect SQL pairs the strict AST-equivalence rate is 0.800 overall and varies predictably with query difficulty and dialect pair, exposing where transpile-success metrics are optimistic; a failure-mode taxonomy further partitions that headline into a large cosmetic-drift bucket an audit pipeline can safely auto-promote and a smaller bucket of genuinely lossy translations that must be reviewed by hand. The contribution is therefore not a new accuracy record but a transparent, reproducible demonstration that deployment-aware, confidence-coupled integration of validation and migration is both feasible and operationally sensible—subject to the clearly stated limits of injected evaluation, which expert-annotated and real-world studies should next address. All code, anomaly masks, and SHA-256-verified datasets reproduce end-to-end from a fixed seed.

The limitations follow directly from the threats above. The three anomaly families with near-zero recall (A2, A3, C3) are limited by feature coverage rather than by the fusion mechanism, and call for richer learnt features—tag-embedding similarity for tag-semantics families and signed bill ratios for sign-violation families. The AST-footprint comparator is intentionally strict and would benefit from a relaxed mode that treats semantically equivalent renderings (such as lateral joins and comma-list `FROM` clauses) as the same node class. More broadly, the synthetic injection should be complemented by expert-annotated, field-discovered errors, and the SQL pipeline should be coupled to schema-level metadata so that AST equivalence can be augmented with execution-time row-equivalence checks against representative seed data. Near-term work targets calibration-aware threshold selection [30], [32] and richer features for the underperforming families; longer-term work targets multi-domain validation beyond the three corpora studied here.

• • •

REFERENCES

[1] T. C. Redman, "The impact of poor data quality on the typical enterprise," Communications of the ACM, vol. 41, no. 2, pp. 79–82, 1998.

[2] L. Ehrlinger and W. Wöß, "A survey of data quality measurement and monitoring tools," Frontiers in Big Data, vol. 5, p. 850611, 2022.

[3] M. Stonebraker and I. F. Ilyas, "Data integration: The current status and the way forward," in IEEE Data Engineering Bulletin, vol. 41, no. 2, 2018, pp. 3–9.

[4] I. F. Ilyas and X. Chu, "Data cleaning," ACM Books, Morgan & Claypool, 2019.

[5] C. Batini, C. Cappiello, C. Francalanci, and A. Maurino, "Methodologies for data quality assessment and improvement," ACM Computing Surveys, vol. 41, no. 3, pp. 16:1–16:52, 2009.

[6] Z. Abedjan, X. Chu, D. Deng, R. C. Fernandez, I. F. Ilyas, M. Ouzzani, P. Papotti, M. Stonebraker, and N. Tang, "Detecting data errors: Where are we and what needs to be done?" in Proceedings of the VLDB Endowment, vol. 9, no. 12, 2016, pp. 993–1004.

[7] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," in Proceedings of the VLDB Endowment, vol. 11, no. 12, 2018, pp. 1781–1794.

[8] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "HoloClean: Holistic data repairs with probabilistic inference," in Proceedings of the VLDB Endowment, vol. 10, no. 11, 2017, pp. 1190–1201.

[9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in Proc. IEEE International Conference on Data Mining (ICDM), 2008, pp. 413–422.

[10] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in Proc. ACM SIGMOD International Conference on Management of Data, 2000, pp. 93–104.

[11] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," ACM Computing Surveys, vol. 41, no. 3, pp. 15:1–15:58, 2009.

[12] L. Ruff, J. R. Kauffmann, R. A. Vandermeulen, G. Montavon, W. Samek, M. Kloft, T. G. Dietterich, and K.-R. Müller, "A unifying review of deep and shallow anomaly detection," Proceedings of the IEEE, vol. 109, no. 5, pp. 756–795, 2021.

[13] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," ACM Computing Surveys, vol. 54, no. 2, pp. 1–38, 2021.

[14] C. C. Aggarwal, Outlier Analysis, 2nd ed. Springer, 2017.

[15] S. Han, X. Hu, H. Huang, M. Jiang, and Y. Zhao, "ADBench: Anomaly detection benchmark," Advances in Neural Information Processing Systems (NeurIPS), vol. 35, pp. 32 142–32 159, 2022.

[16] Y. Zhao, Z. Nasrullah, and Z. Li, "PyOD: A python toolbox for scalable outlier detection," Journal of Machine Learning Research, vol. 20, no. 96, pp. 1–7, 2019.

[17] D. H. Wolpert, "Stacked generalization," Neural Networks, vol. 5, no. 2, pp. 241–259, 1992.

[18] C. C. Aggarwal and S. Sathe, Outlier Ensembles: An Introduction. Springer, 2017.

[19] O. Sagi and L. Rokach, "Ensemble learning: A survey," WIREs Data Mining and Knowledge Discovery, vol. 8, no. 4, p. e1249, 2018.

[20] E. Rahm and P. A. Bernstein, "A survey of approaches to automatic schema matching," The VLDB Journal, vol. 10, no. 4, pp. 334–350, 2001.

[21] T. Mao and SQLGlot Contributors, "SQLGlot: A no-dependency SQL parser, transpiler, optimizer, and engine," https://github.com/tobymao/sqlglot, 2024.

[22] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task," in Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2018, pp. 3911–3921.

[23] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo et al., "Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-sqls (BIRD)," Advances in Neural Information Processing Systems (NeurIPS), vol. 36, pp. 42 330–42 357, 2023.

[24] G. Katsogiannis-Meimarakis and G. Koutrika, "A survey on deep learning approaches for text-to-sql," The VLDB Journal, vol. 32, no. 4, pp. 905–936, 2023.

[25] X. Liu, S. Shen, B. Li, P. Ma, R. Jiang, Y. Luo, Y. Zhang, J. Fan, G. Li, and N. Tang, "A survey of NL2SQL with large language models: Where are we, and where are we going?" arXiv preprint arXiv:2408.05109, 2024.

[26] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," Advances in Neural Information Processing Systems (NeurIPS), vol. 28, 2015.

[27] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," ACM Computing Surveys, vol. 55, no. 6, pp. 1–29, 2023.

[28] D. Kaur, S. Uslu, K. J. Rittichier, and A. Durresi, "Trustworthy artificial intelligence: A review," ACM Computing Surveys, vol. 55, no. 2, pp. 1–38, 2023.

[29] B. Li, P. Qi, B. Liu, S. Di, J. Liu, J. Pei, J. Yi, and B. Zhou, "Trustworthy AI: From principles to practices," ACM Computing Surveys, vol. 55, no. 9, pp. 1–46, 2023.

[30] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. International Conference on Machine Learning (ICML), 2017, pp. 1321–1330.
[31] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.
[32] M. Abdar, F. Pourpanah, S. Hussain, D. Rezazadegan, L. Liu, M. Ghavamzadeh, P. Fieguth, X. Cao, A. Khosravi, U. R. Acharya, V. Makarenkov, and S. Nahavandi, "A review of uncertainty quantification in deep learning: Techniques, applications and challenges," Information Fusion, vol. 76, pp. 243–297, 2021.
[33] P. J. Rousseeuw and C. Croux, "Alternatives to the median absolute deviation," Journal of the American Statistical Association, vol. 88, no. 424, pp. 1273–1283, 1993.
[34] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," Journal of Machine Learning Research, vol. 22, no. 164, pp. 1–20, 2021.
[35] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
[36] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith et al., "Array programming with NumPy," Nature, vol. 585, no. 7825, pp. 357–362, 2020.

UPASANA BHAUMIK is an undergraduate student in Computer Science and Engineering with a specialization in Bioinformatics at Vellore Institute of Technology, India. She actively builds AI-driven and data-centric applications, combining machine learning, data engineering, and full-stack development to solve real-world problems. Her work includes developing intelligent systems for data analysis, predictive modeling, and interactive applications powered by large language models. She is particularly interested in leveraging AI for bioinformatics and healthcare, along with advancing data quality and automation in large-scale systems.

BERLIN M A received the master's degree in computer science engineering from National Engineering College, Tamil Nadu, and the Ph.D. degree in computer science engineering from Anna University, Chennai. Currently, she is an Associate Professor with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore. Her research interests include ad hoc networks, IoT and machine learning. She has over 20 years of experience in teaching and research. She has published many papers in international journals and conferences on vehicular ad hoc networks, machine learning and deep learning. She has published few patents on IoT and artificial intelligence.